AI Browser Automation Tool

Research & Build Guide — a client-side AI assistant that understands pages, fills any form, and summarizes content

Version 2.0 · 29 October 2026 — adds: every website without ARIA, lightweight design, edge cases, every browser

What this document is

One place for everything learned in the research: what the product should do, what existing tools teach us, which technologies to use, how each hard problem is solved, what can still go wrong, and the order to build it in.

It is written to be read top to bottom by anyone on the team. Technical detail is kept where it matters for building. Supporting evidence and source links live in the research files listed in the appendix.

The one-paragraph answer

Build **one shared engine** with a thin extension for each browser (Chrome first, then Edge, Firefox, Safari), running entirely on the user's device and loading **only when the user asks**. It reads any page from its HTML structure and layout, so it does not need accessibility labels, and switches to a visual fallback only when there is no usable HTML. It uses the **cheapest AI that can answer** (rules first, then on-device models, cloud only if the user allows), and fills fields with **its own code**, never by letting AI run code. Personal data stays **encrypted on the device** and is never sent to any AI. Every successful fill is saved as replayable steps that **repair themselves** when a site changes, and the extension **always asks before submitting, paying or sending**.

Contents

1. The product in plain words 4

1.1 Requirements we agreed on . . . 4

2. What existing tools teach us 4

2.1 The common pattern everyone converged on . . . 5

3. Key decisions 5

4. Architecture 6

4.1 What each part does . . . 6

5. How each part works 7

5.1 Reading and understanding the page . . . 7

5.2 Understanding each field — the AI ladder . . . 7

5.3 Memory (all on the device) . . . 7

5.4 Filling every input type . . . 8

5.5 Self-healing saved steps . . . 9

5.6 Saving progress safely . . . 9

5.7 Encrypted local vault . . . 10

5.8 Safety gates . . . 10

5.9 Permissions . . . 10

5.10 User interface . . . 10

6. Works on every website — without relying on ARIA 11

6.1 Finding clickable things . . . 11

6.2 Finding labels without tags . . . 11

6.3 When the page has no usable HTML . . . 11

7. Lightweight and fast 12

7.1 Design rules . . . 12

7.2 Performance budget . . . 12

8. Works in every browser 13

8.1 What each browser gets . . . 13

8.2 Rollout order . . . 14

8.3 Engine differences to handle . . . 14

9. Edge cases: websites and situations 14

10. Technology stack 15

11. Issues and how we solve them 15

12. Corrections to earlier research 17

13. Build plan **17**

13.1 Measure from day one 18

14. Honest limits and open questions **18**

14.1 Test before committing 18

Appendix A. Glossary **18**

Appendix B. Research files and key sources **19**

1. The product in plain words

The goal is an AI assistant that lives in the browser and helps with any web page:

- **Understand the page** — what it is for, what you can do on it, what forms and tables it has.
- **Summarize** — like the Gemini page icon, but with structured output (purpose, key facts, actions, risks).
- **Fill forms automatically** from information the user has saved once (name, address, work history, documents).
- **Extract data** — pick a region of the page and turn it into a table or JSON.
- **Select and prompt** — click any element on screen and tell the AI what to do with it.
- **Repeat tasks** — record a task once and replay it later, even after the site changes.

1.1 Requirements we agreed on

Requirement	What it means in practice
Works on any site	React, Vue, Angular, Svelte, plain HTML/CSS, PHP pages, jQuery, web components — no per-framework product.
Fills every input type	Text, dropdowns, searchable comboboxes, date pickers, masked inputs (phone, card), rich-text editors, file uploads, checkboxes, radios, sliders, inside iframes and shadow DOM.
Fast and cheap	Most fills use no AI at all. AI is called only for what rules cannot decide, and its answers are cached.
Client-side only	No server-hosted browsers, no backend database. Everything runs in the user's browser or on their computer.
Personal data stays local	Profile is encrypted on the device. AI sees field labels and profile field names, never the values.
Asks before risky actions	Submit, pay, send, delete, passwords and card fields always need the user's click to confirm.
Survives site changes	Detects that a page changed, retries, and repairs only the steps that broke.
Survives browser limits	Saves progress after every step; never depends on Chrome's background process staying alive.
Works on badly built sites	Does not depend on accessibility labels; reads structure, layout and styles; visual fallback for canvas pages.
Every browser	Chrome, Edge, Brave, Opera, Vivaldi, Arc, Firefox (desktop + Android), Safari (Mac + iPhone), with a bookmarklet fallback.
Lightweight	Zero cost on pages where it is not used; small engine; no automation framework.

2. What existing tools teach us

We studied open-source agents, commercial products, password managers, job-application autofill tools, and coding tools with click-to-edit. The table lists what to copy and what to avoid.

Tool	How it works	Take	Avoid
Nanobrowser (open-source Chrome extension)	Side-panel agent on a deterministic browser-control layer using chrome.debugger.	Element IDs bound to exact elements; checks visible / stable / enabled / not covered before acting; Ask vs Auto modes; per-site rules; local models via Ollama.	Relying on the AI to notice a sensitive action. Debugger-only control (shows a warning bar).
Bitwarden (password manager)	Content script sets values and fires events; encrypted vault with session key.	Event order for filling; how it locks the vault; shadow-DOM tracking; passkey unlock.	—

Tool	How it works	Take	Avoid
Chrome's own autofill	Heuristics + autocomplete hints; form/field signatures.	Event order focus → keydown → value → input → change → keyup → blur; signature hashing to recognise forms.	—
job_app_filler, Autofill-Jobs	Open-source Workday/Greenhouse fillers.	Site adapters; filling one field at a time through a queue; calling React handlers directly.	—
Alibaba Page Agent	JavaScript inside the page, text-only DOM, no screenshots.	Text-indexed page snapshot.	Only covers one page.
Browser Use	Python library driving Chrome over CDP.	Snapshot with occlusion filtering; loop detection; reference for a desktop companion.	Server/Python in the core product.
Stagehand, Skyvern, workflow-use	Server-side: AI once, then cache or generate code.	The idea: AI once → save → replay → AI only on failure.	Their server caches and generated code (not allowed in extensions).
Gemini in Chrome	Built into Chrome.	Security design: origin limits, action checker, confirmations.	—
Antigravity IDE	Built-in browser agent via Chrome DevTools MCP; screenshots on request; video recordings.	Accessibility snapshot + on-demand screenshots; local recordings for audit.	Debug-port model (Chrome 136+ blocks it on the normal profile).
React Grab, v0, Onlook, Lovable	Click a component, then prompt; map element to source code.	Picker overlay that captures element fingerprint + subtree + screenshot crop.	Source mapping only works in dev builds — dev mode only.
acebuilder.ai	Aceternity's AI landing-page builder.	Little to learn — selection method not documented.	—

2.1 The common pattern everyone converged on

- Read the page as a **compact accessibility/DOM tree with stable references**; screenshots only as a backup.
- Let the AI **decide**, and let normal code **act**.
- **Save** what worked so the next run needs no AI.
- Put safety in **code** (confirmations, origin limits), because AI-side defenses are not reliable.

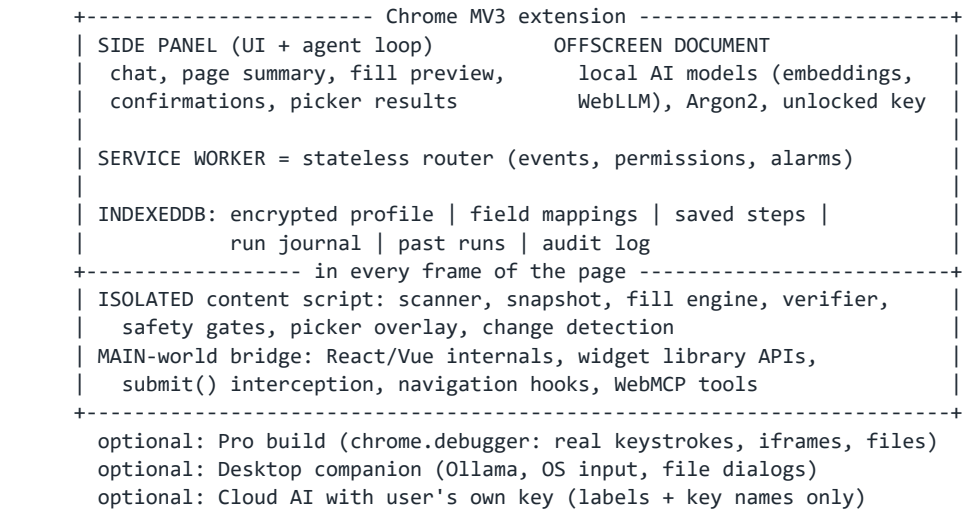
3. Key decisions

#	Decision	Why
1	**One shared engine + thin browser extension per browser**, Chrome first.	Extensions are the only option that works in the user's own logged-in tabs with no server; the shared engine makes every browser cheap to add.
1b	**Nothing loads until the user asks**; no automation framework (Playwright only for testing).	Zero slowdown on pages where the tool is not used.
1c	**Read pages from HTML structure, layout and styles**, not ARIA; visual fallback only when needed.	Many sites have no accessibility labels.
2	**No debugger permission** in the main version.	Cannot be made optional, shows a scary install warning and a permanent 'debugging this browser' bar, slows store review.

#	Decision	Why
3	Trusted (real-keystroke) input goes in a separate **Pro build** or the **desktop companion**.	Keeps the main install light; power users opt in.
4	**No server anywhere.** Cloud AI only with the user's own API key, per site, opt-in.	Client-side requirement; privacy; no hosting cost.
5	**AI returns JSON, extension code executes it.**	Chrome Web Store bans running remote or generated code.
6	**Personal values never reach any AI.**	Prompt injection cannot leak what the model never saw.
7	**Record once, replay without AI, repair only broken steps.**	Speed, cost and reliability.
8	**Progress saved to IndexedDB after every step.**	Chrome stops the background process after 30 s idle; session storage is wiped on restart.
9	**Deterministic confirmation gates** before submit/pay/send.	The AI may not notice a risky action; code always does.
10	Phase 2: **desktop companion** + **Firefox port**. Defer Safari, Electron/Tauri.	Companion unlocks file paths, OS input, local models. Electron forces users to log in again.

4. Architecture

The extension is split into parts that each have one job. The background service worker only passes messages, because Chrome can stop it at any time.



4.1 What each part does

Part	Job	Notes
Side panel	User interface and the main AI loop.	Stays alive while open. Opens only from a user click/shortcut.
Offscreen document	Runs local AI models and holds the unlocked vault key in memory.	One per browser profile. Needs bundled WASM (no downloaded code).
Service worker	Routes messages, handles permissions, alarms, navigation events.	Stops after 30 s idle. Holds no state.
Content script (isolated)	Reads the page, fills fields, verifies, runs safety gates.	Runs in every frame. Shares the page DOM but not its JavaScript.
MAIN-world bridge	Calls page JavaScript: React props, flatpickr, CKEditor, TinyMCE, WebMCP.	The page could tamper with it — never give it secrets.

Part	Job	Notes
IndexedDB	All durable data.	Survives restarts and updates.
storage.session	Hot state and the session key only.	In memory; cleared on restart or update (locks the vault — by design).

5. How each part works

5.1 Reading and understanding the page

1. If the site exposes **WebMCP** tools (Chrome's new standard for sites to describe their forms to AI), use them first — most reliable.
2. Otherwise build a **compact snapshot**: every interactive element with a short number, its role, accessible name, label and state. Format example: `[12, "tbox", "Email", {"req":1}]`.
3. Drop invisible elements — **except** real inputs hidden behind styled checkboxes, radios and dropdowns.
4. Remove hidden or near-invisible text (a common place to hide malicious instructions).
5. **Shadow DOM:** walk open roots; use `chrome.dom.openOrClosedShadowRoot(el)` for closed ones (Chrome 88+).
6. **Iframes:** a content script in every frame reports to the top frame. Cross-site iframes need permission for that site.
7. **Canvas apps** (Sheets, Figma) or pages with poor markup: take a screenshot and number the regions ('set-of-marks').
8. **Summaries:** extract readable text, run Chrome's built-in Summarizer (on-device, Chrome 138+), fall back to cloud for long pages.

5.2 Understanding each field — the AI ladder

Each step is tried in order. The tool stops at the first confident answer, so most fields never reach an AI.

Step	Method	Cost / privacy	Accept when
1	Rules: `autocomplete` hints, input type, label/name patterns (many languages), saved mapping for this form.	Free, instant	score ≥ 0.9
2	Small local matching model (MiniLM / bge-small) comparing label text to profile field descriptions.	Free, on device, ~25–35 MB	top ≥ 0.75 and clear margin
3	Chrome built-in **Gemini Nano** (Prompt API, extensions, Chrome 138+) with a fixed list of allowed answers.	Free, on device; needs strong PC	agrees with step 2 or confidence ≥ 0.7
4	Local Ollama / LM Studio if running.	Free, on device	valid JSON answer
5	WebLLM in the browser (opt-in download ~2 GB).	Free, on device	valid JSON answer
6	Cloud AI with user's own key (opt-in per site).	Paid; sends labels + key names only	valid JSON answer

- AI output is always `{ref, profileKey or unknown, confidence}`.
- Confidence decides the action: **fill automatically**, **suggest**, or **ask the user**.
- If the profile has no value, the tool asks and saves the answer. It never invents data.
- Accepted answers are cached by form/field signature, so the next visit is step 1.

5.3 Memory (all on the device)

Memory	Holds	Used for
Profile	Typed fields aligned to browser autofill names, plus custom fields, synonyms, formats. Each field has a sensitivity tier and a 'never autofill' switch.	The values to fill.
Site memory	Learned field-to-profile mappings and user corrections, by form signature.	Skipping AI on known forms.
Saved steps	Recorded tasks as replayable steps.	Repeat tasks without AI.
Past runs	What happened, what failed, what was repaired.	Resume, debugging, 'why did this fail'.

5.4 Filling every input type

A **fill engine** detects the widget type, picks a handler, fills, then reads the value back. If the value did not stick, it tries a stronger method.

Standard event order (from Chrome's own autofill)

```
focus -> keydown -> set value -> input -> change -> keyup -> blur
```

Escalation per field (3–5 second budget)

1. Widget-specific handler (from the content script).
2. Generic handler based on ARIA roles or native element type.
3. Type character by character with `execCommand('insertText')`.
4. Call the widget library's own API in the page (MAIN world).
5. Real keystrokes/clicks through the debugger (Pro build or companion).
6. Give up gracefully: highlight the field and ask the user.

How each widget type is handled

Widget	Detect by	How to fill
Text (React, Vue, Angular, Svelte, plain)	input / textarea	Set value, fire `input` and `change`, end with `blur` (Angular needs blur to mark 'touched').
Native select	select	Match option by text then value; set; fire input + change.
Custom dropdown / combobox (react-select, MUI, AntD, Radix, Headless UI, Select2, Chosen, Angular Material)	`role=combobox`, `aria-haspopup=listbox`, class prefixes	Full pointer sequence to open → wait for listbox → type to filter (needed for long virtual lists) → click matching option → verify displayed value. Radix needs real PointerEvents with pointerType 'mouse'.
Shortcuts	MUI hidden native input; Select2/Chosen underlying select	Set the hidden input / native select and fire change — the widget syncs itself.
Native date/time	type=date etc.	Exact format: `yyyy-mm-dd`, `yyyy-mm`, `HH:mm`; verify validity.
Date pickers	MUI X, AntD, react-datepicker, flatpickr	MUI X: set hidden input. AntD: type exact format then Enter (blur does not save). flatpickr: call `_flatpickr.setDate`. Fallback: click calendar cells.
Masked inputs	imask, cleave, react-input-mask, Inputmask	Type raw characters one at a time; verify after removing mask characters.
Rich text	ProseMirror, Lexical, Draft.js, Quill, CKEditor, TinyMCE (iframe)	insertText, or a synthetic paste event, or the editor's own API.
File upload	input type=file, drop zones	Build a File, assign via `DataTransfer`, fire change; drop zones get a synthetic drop event.

Widget	Detect by	How to fill
Checkbox / radio	checkbox, radio, ARIA switch	Use `click()` (never set checked silently); match radios by label text.
Slider	range, ARIA slider	Set value for native; arrow keys for ARIA sliders.
Site-specific	Workday, Greenhouse, Lever, iCIMS, Ashby	Dedicated adapters from open-source job fillers.

Rules

- Fill one field at a time through a queue — dropdown menus clash if opened together.
- Re-scan after each fill; new questions often appear.
- Scroll into view and wait until the element stops moving before acting.
- A field is done only when the read-back matches **and** no error message appeared.

5.5 Self-healing saved steps

Each saved step stores several ways to find its element, so it survives redesigns.

- **Form signature** (Chrome autofill method): a hash of site + form name + field names. A second 'shape' signature uses field types only, to detect renamed fields.
- **Ranked locators** (Playwright order): test id, role + name, label, placeholder, id, robust XPath.
- **Fingerprint** of the element (Similo++ method): name, visible text, type, attributes, nearby text, position, size, XPath, ancestor path.

When a step breaks

1. Compare the page's form signature. Same form → use stored locators. Fields renamed → only the missing ones are suspect. Form rebuilt → check each step.
2. Try ranked locators; accept a unique match whose fingerprint still scores high.
3. Otherwise score every candidate element and pick the best if it clearly beats the second best.
4. Still unclear → ask the AI to choose among the top 3 (labels only), or ask the user.
5. **Never** auto-repair a step that submits, pays or sends.
6. Save the repair **only after** the step succeeds.
7. Two failed repairs → hand that one step to the AI agent.

Stuck detection

- Hash each action and the page state. Same action or unchanged page repeatedly → nudge at 3, stop at 5.
- Retry with growing delays (250 ms, 500 ms, 1 s) only for temporary errors.
- Keep checkpoints; on a deep stall, go back to the last good step.

5.6 Saving progress safely

1. Run the AI loop in the side panel or offscreen document, not the service worker.
2. Before each action, write the step as `applying`; after, `applied`; after reading it back, `verified`.
3. After a restart, for any step still 'applying', look at the page: if the field already has the value, mark it done; otherwise redo it.
4. Only repeat actions that are safe to repeat. 'Click Add row' first checks how many rows exist.
5. **Never re-submit a form automatically after a crash.** Ask the user.

5.7 Encrypted local vault

- A random 256-bit data key encrypts each record with AES-GCM.
- The data key is wrapped by a key from the user's **password** (Argon2id, or PBKDF2-SHA256 with ≥ 600,000 iterations) or a **passkey** (WebAuthn PRF). Optional short PIN for quick unlock.
- Unlocked key lives only in memory; auto-lock after idle (default 15 min), on screen lock, and on browser close.
- Page scripts get only the values for one fill, and only if: the user started the fill on this tab, the frame is the same site, and the field's tier allows it.

Tier	Examples	Rule
T0	Name, city	Unlock once per session.
T1	Email, phone, address	Unlock once per session.
T2	Government ID, date of birth	Re-confirm (PIN or passkey) every fill.
T3	Card, bank account	Re-confirm every fill; never sent to any AI; only into card fields or after explicit confirmation.

5.8 Safety gates

These run in code on every action, whatever the AI says. Any match pauses for the user's click.

- Any form submit (caught at capture phase and by wrapping `form.submit()` in the page).
- Any password field; any `autocomplete` starting with `cc-`, `transaction-` or `one-time-code`.
- Buttons whose text matches pay, place order, purchase, buy, submit, send, confirm, transfer, delete, sign (in several languages).
- Forms that post to a different site.
- No navigation or network calls to new sites during a fill; the agent never makes network requests for the page.
- CAPTCHA or 2FA detected → hand to the user. Never solve CAPTCHAs.
- Every action logged in an encrypted, tamper-evident audit log (names of fields filled, never values).

5.9 Permissions

Permission	Warning shown?	Why
`activeTab`, `scripting`, `sidePanel`, `offscreen`	No	Core features.
`storage`, `unlimitedStorage`, `alarms`, `idle`	No	Data, auto-lock.
`optional_host_permissions: https://*/*`	Asked per site	User grants each site when first used.
`webNavigation`	'Read your browsing history'	Detect page changes in single-page apps (optional).
`nativeMessaging`	Yes, but optional	Desktop companion.
`debugger`	Strong warning + bar	Pro build only.

Content Security Policy for extension pages: `script-src 'self' 'wasm-unsafe-eval'; object-src 'self';` — allows bundled WASM, nothing remote.

5.10 User interface

- Side panel with chat and page summary.
- Colored highlights on each field showing which profile field will be used and how confident the tool is.
- Preview / accept panel before anything is filled; per-field on/off; undo.
- Per-site autonomy: suggest only, fill, or fill and submit.
- Element picker for 'select and prompt': hover highlights, click captures the element, its section and a screenshot crop.
- Popup-window mode for browsers without a side panel (Opera, Vivaldi) or when the side panel squeezes the page into a mobile layout.

6. Works on every website — without relying on ARIA

> **Why this works even on badly built sites**
>
> ARIA labels are optional extras that many sites skip. But every HTML page has a **DOM** (the elements the browser draws), **layout boxes** (where each thing is), **computed style** (for example the hand-shaped cursor) and **hit-testing** (what is actually on top at a point). Vimium, Tridactyl, Browser Use and Bitwarden all rely on these, not on ARIA. ARIA, when present, only raises confidence.

6.1 Finding clickable things

Signal	Strength	Example
Native control (input, select, textarea, button, a)	Strong	Any form field
Editable region, keyboard-focusable (`tabindex>=0`)	Strong	Rich-text editors, custom widgets
Click handler found	Strong	`onclick`, Google `jsaction`, React `__reactProps$` onClick, Vue `_vei`, Svelte events, jQuery events
Hand cursor at the top of a region	Medium	A `div` styled as a button; nested spans ignored
Icon-sized element (10–50 px) with svg/img	Medium	Close, search, menu icons
Meaningful class words after removing random hashes	Medium	`btn`, `menu`, `close` kept; `css-1x9ab` dropped
Disabled, inert, pointer-events none, covered, under 4 px	Excludes	Greyed-out buttons, hidden traps

Optional **deep scan**: on sites the user marks 'always help here', a tiny script loads at page start and records which elements receive click listeners — the strongest signal for 'div soup' sites. Off by default to keep zero cost on other sites.

6.2 Finding labels without tags

1. Standard sources: linked label, `aria-label`, title, placeholder, alt, SVG title.
2. Chrome autofill's own rules: text just before the field, div-based layouts, table rows and columns, definition lists, fieldset legend.
3. Position: nearest short text on the same row to the left, or just above; each text belongs to only one field.
4. Field attributes after cleanup: `name`, `id`, `autocomplete`, `data-testid` split into words.
5. Last resort: read the text near the field from a screenshot (OCR).

6.3 When the page has no usable HTML

Page type	First try	Then
Flutter web apps	Click the hidden 'enable accessibility' element to build a semantic tree	Visual fallback
Google Sheets / Docs	Use the DOM parts: Name Box, formula bar, menus, hidden text box	Visual fallback
PDFs in the browser viewer	Download the PDF and read it with pdf.js	Screenshot + OCR
Canvas, WebGL, Figma, games	Toolbars and panels are still HTML	Visual fallback
Iframes we cannot enter	Ask permission for that site	Visual fallback

Visual fallback (only when needed): screenshot (Chrome allows 2 per second) → small element detector (about 3–6 MB, ~30 ms per frame on an M1) → text recognition (PP-OCRv5 mobile, about 20 MB, ~100–300 ms) → Gemini Nano or cloud AI only to name an unlabeled icon. Avoid OmniParser for a closed product (AGPL licence).

Expected coverage (estimate): the HTML layers find about 90–95% of controls on normal sites, including badly built ones; the visual layer covers most of the rest. Actions needing real user input and fine canvas drawing stay with the user.

7. Lightweight and fast

Measured on real extensions: Honey adds 825 ms of CPU per shopping page, Evernote 368 ms. Grammarly went from about 500 ms to about 100 ms by loading its full engine only where needed. Bitwarden slows some sites because it watches every page for changes all the time. We avoid both mistakes.

7.1 Design rules

- **Nothing runs until the user asks.** No script is placed on every page. The engine (under 50 KB) loads when the user clicks the toolbar button, presses the shortcut, uses the right-click menu or opens the side panel.
- **Watch only while open.** Page-change watching starts with the assistant and stops when it closes; changes are processed in batches after 100–250 ms.
- **Fast scanning.** Use the browser's built-in element search, read all positions and styles in one pass, write later; split big pages into ~8 ms chunks so the page never freezes.
- **No framework in the page.** Plain TypeScript in the page; Preact or Lit only for the small overlay; the rich interface lives in the side panel.
- **Small AI requests.** One short line per field (~10–25 tokens), one batched request, streamed so fields fill as answers arrive.
- **No AI on repeat visits.** Saved mappings fill known forms instantly.
- **Models are optional and unloaded when idle** (after ~2 minutes). Rules and dictionaries handle most fields without any model.

7.2 Performance budget

What	Target
Cost per tab before use	0 — no script, no CPU, no memory
Engine size	under 50 KB (under 15 KB compressed)
Click to engine ready	under 50 ms
Scan a normal form page (2–5k elements)	under 30 ms

What	Target
Scan a huge page (50k elements)	under 150 ms, never blocking more than 10 ms at once
First field filled from saved mapping	under 100 ms
First field filled with cloud AI	under 1.5 s
AI calls on a repeat visit	0
Memory while active	page script under 5 MB; side panel under 50 MB; local model host under 150 MB
Slowdown of the page after filling	under 10 ms

Several targets are engineering estimates; they become release gates once measured with Chrome Task Manager, DevTools Performance and an A/B test on ~20 heavy sites.

8. Works in every browser

One shared engine runs everywhere; only a thin layer changes per browser. Missing features are detected at start-up and switched off gracefully.

```
core/   engine: page reader, field matcher, filler, encrypted vault (plain TypeScript, no browser APIs)
ai/     model providers chosen at runtime: Chrome Gemini Nano, Edge Phi-4-mini,
        WebGPU/WASM models, Apple Foundation Models, desktop companion, rules only
shell/  one thin layer per browser, built with WXT:
        Chromium -> side panel + offscreen page      Firefox -> sidebar + event page
        Safari   -> popup + in-page overlay         Bookmarklet / userscript -> page only
```

8.1 What each browser gets

Browser	Panel	Local AI	Hidden shadow parts	Pro (real keystrokes)	Store
Chrome, Edge	Side panel	Gemini Nano / Phi-4-mini / WebGPU	Yes	Yes	Chrome Web Store, Edge Add-ons
Brave, Vivaldi, Arc, Yandex, Opera	Side panel (Opera: sidebar)	WebGPU	Yes	Yes	Install from Chrome Web Store
Firefox desktop (+ LibreWolf, Zen, Waterfox)	Sidebar	WebGPU (Windows) / WASM	Yes	No	Firefox Add-ons (AMO)
Firefox Android	Overlay / popup	Rules + tiny models	Yes	No	AMO
Safari macOS	Popup + overlay	Apple Foundation Models / WebGPU	Opt-in per site	No	App Store ($99/yr)
Safari iPhone / iPad	Overlay	Apple Foundation Models	Opt-in per site	No	App Store
Orion	Popup	WASM	No	No	Installs Chrome & Firefox extensions
DuckDuckGo, Chrome Android, locked-down PCs	Overlay	None (rules)	No	No	Bookmarklet / userscript
Samsung Internet	—	—	—	—	Closed beta; later
Tor Browser	—	—	—	—	Not supported (extensions hurt anonymity)

8.2 Rollout order

1. Chrome Web Store build — also covers Brave, Vivaldi, Arc, Yandex and Opera (about 70%+ of desktop users).
2. Edge Add-ons — same build plus Edge's on-device model.
3. Firefox desktop and Android — sidebar version, Android overlay; AMO requires source upload.
4. Safari macOS and iOS — Apple developer account, Xcode build, overlay UI, Apple on-device AI.
5. Opera store listing and Orion testing.
6. Bookmarklet / userscript version for browsers without extensions.
7. Optional desktop companion and Samsung Internet.

8.3 Engine differences to handle

- Firefox hides page JavaScript objects from extension scripts — framework fills run in the page's own context there.
- Safari has no way to open closed shadow parts — an opt-in early script records them instead.
- `scheduler.yield` is missing in Safari — fall back to `setTimeout`.
- Firefox and Safari need the permission request called directly inside the click handler (no waiting first).
- iPhone file pickers cannot be automated — file fields are marked 'do it yourself' there.
- Chrome on Android still has no extensions; DuckDuckGo allows none — bookmarklet fallback.

9. Edge cases: websites and situations

About 90 cases were catalogued (full list in research file 09). The most important ones:

Situation	What we do
Browser's own pages, Chrome Web Store, admin-blocked sites	Show 'this page can't be read'; never fail silently. Summaries still work on pasted text.
PDF in the browser viewer	Download and read with pdf.js.
Hidden 'trap' fields that catch bots (honeypots)	Only fill fields a person can see; skip anything hidden, off-screen, tiny or named like 'leave blank'.
Never-fill fields	Passwords, card fields in payment frames, one-time codes, CAPTCHAs, consent / terms / marketing checkboxes — hand to the user.
Pages that re-render after loading (React SSR, Next.js)	Wait until the page settles; re-check each value after 1 s and refill once.
Single-page apps and partial page swaps (Turbo, HTMX, PJAX)	Listen for their navigation events and re-scan.
ASP.NET pages that reload when a dropdown changes	Fill in page order; after such a field, wait for reload, re-scan, continue.
Multi-page wizards, one-question-per-screen forms	Keep progress across pages; fill, press Next, wait, re-scan.
Very large pages, long virtual lists and grids	Chunked scanning with caps; scroll the list step by step.
Cookie banners, sticky headers, pop-up dialogs	Check what is on top before clicking; act only inside the topmost dialog.
Strict security policies (Google, GitHub)	No innerHTML or eval; UI in the side panel or a closed shadow root.
Sites that patch browser functions	Do work in the extension's clean context; use page context only to read framework data.

Situation	What we do
Other extensions in the same field (Grammarly, password managers)	Leave credentials to them; ignore their added elements; re-check values.
Google Translate rewriting the page	Match fields by attributes, not translated text.
Extension updated or browser crashed mid-task	Old page script removes itself; task resumes from the last saved step; never re-submits.
Session timed out mid-form	Keep entered values; offer to refill after login.
Low-end laptop (4 GB RAM)	Smaller scan limits, no on-device models, plain UI.
Private windows, enterprise policies, file pages	Detect and explain the setting the user or admin must change.

Cannot be fixed (show a clear message): browser internal pages, Chrome Web Store, admin-blocked sites, native Apple Pay / Google Pay sheets, passkey prompts, Chrome on Android (no extensions).

10. Technology stack

Area	Use	Notes
Language	TypeScript	Shared types for snapshots, steps, profile.
Extension	WXT build tool; Manifest V3; `browser.*` API with promises	One codebase → Chrome, Edge, Firefox, Safari, Opera.
Visual fallback	onnxruntime-web + nano YOLO GUI detector; PP-OCRv5 mobile; pdf.js	Loaded only on canvas / PDF pages.
Page reading	Own snapshotter (DOM + ARIA), TreeWalker, MutationObserver, Navigation API	Ideas from Browser Use buildDomTree, Playwright aria snapshots.
Readable text	Readability-style extraction	For summaries.
On-device AI	Chrome Prompt API (Gemini Nano), Summarizer API	Free; hardware-dependent.
Local embeddings	Transformers.js + MiniLM / bge-small (WebGPU, WASM fallback)	Runs in offscreen document.
Optional local LLM	WebLLM (Qwen3-1.7B) or Ollama / LM Studio	User opt-in.
Cloud AI (optional)	Any provider via user's own key	Labels + key names only; prompt caching.
Storage	IndexedDB (via a small wrapper, e.g. idb), chrome.storage	Journal, profile, steps, logs.
Crypto	WebCrypto AES-GCM, PBKDF2; Argon2id via hash-wasm; WebAuthn PRF	Bitwarden-style locking.
Fuzzy matching	Own Levenshtein / Jaccard, Similo++ weights	For self-healing and option matching.
Phone / format checks	libphonenumber-js, Intl APIs	Validate before filling.
UI	React or Svelte in side panel; Shadow-DOM-isolated overlay in page	Overlay must not inherit site CSS.
Testing	Playwright (in CI only) against a benchmark of ~50 real forms	Accuracy, wrong-fill rate, speed, cost.
Companion (phase 2)	Native messaging host (Node or Rust), optional Playwright/Ollama	Signed installer.

11. Issues and how we solve them

Severity: **Critical** = product fails without a fix; **High** = major quality or trust problem; **Medium** = limited impact.

Issue	Severity	Solution
Chrome stops the background process after 30 s idle	Critical	Stateless service worker; AI loop in side panel/offscreen; journal in IndexedDB every step; resume by checking the page.
React/Vue/Angular ignore values set by script	Critical	Set value, then fire input/change/blur in Chrome-autofill order; read back after a tick.
Custom dropdowns, date pickers, masked inputs	Critical	Widget handlers per library + generic ARIA handler + escalation ladder; type to filter virtual lists.
Matching a field to the right profile data	Critical	Rules → local model → Nano → cloud; only allowed answers; per-site learning from corrections.
AI inventing values	Critical	Fixed answer list; confidence threshold; ask instead of guess; read-back and format checks; no auto-submit by default.
Personal data leaking to AI or other sites	Critical	Never send values; bind fills to the site where the user started; card data never to AI; encrypted vault.
Malicious instructions hidden in pages (prompt injection)	Critical	Strip hidden text; mark page text as untrusted; code-level confirmations; no new sites mid-task. Reduces risk, does not remove it.
Debugger warning bar and permission	High	Not in main build; Pro build or companion only.
Chrome Web Store bans remote code	High	AI returns JSON; bundled code executes; never eval.
Shadow DOM (open and closed)	High	TreeWalker + `chrome.dom.openOrClosedShadowRoot`; locators store host chain.
Cross-site iframes	High	Content script in all frames + per-site permission; Pro build uses CDP flat sessions (Chrome 125+), re-attached per level.
Payment iframes (Stripe, Adyen)	High	Fill only from the card vault with explicit confirmation; never from AI output.
Multi-step wizard forms	High	Each step is its own form signature in a saved sequence; pause on review/submit pages.
Sites changing their layout	High	Signatures + ranked locators + fingerprints; repair only broken steps; save repair only after success.
Pages still loading or moving	High	Wait for: element stable for 2 frames, no DOM changes for 200 ms, network quiet for 500 ms, nothing covering it.
Different screen sizes; side panel squeezes the page	High	Act on structure, not pixels; re-find elements after resize; popup-window mode.
CAPTCHA and bot detection	High	Detect and hand to user; never solve or evade (terms of service, store review).
Poor accessibility markup	High	Mix ARIA with DOM clues (pointer cursor, click handlers, tabindex); screenshot fallback.
Agent stuck in a loop	Medium	Action/page hashing; nudge at 3, stop at 5; checkpoints; backoff.
Network or server errors	Medium	Retry with backoff; reload on error pages; tell the AI retry is allowed.
Canvas apps (Docs, Sheets, Figma)	Medium	Screenshot + numbered regions; keyboard actions; app APIs where possible.
File upload needs real path for trusted method	Medium	Use DataTransfer (works for most); CDP file set needs 'Allow access to file URLs' + a downloaded file — last resort.
Gemini Nano needs a strong PC	Medium	Feature-detect; fall back to local embeddings and optional cloud.
Opera / Vivaldi side panel gaps	Medium	Feature-detect; popup window fallback.

Issue	Severity	Solution
Legal: GDPR, site terms, store Limited Use policy	High	Privacy policy; disclose AI use; user-started runs only; per-site deny list; legal review.
Sites with no accessibility labels ('div soup')	Critical	Read structure, cursor, handlers, layout; Chromium label rules + position; optional deep scan; visual fallback.
Extension slowing every page	High	No script until invoked; watch only while open; chunked scans; size budget enforced in CI.
Firefox / Safari missing Chrome features	High	Shared engine + per-browser shell; feature detection; graceful degradation table.
Hidden bot-trap fields	High	Fill only fields a person can see.
Chrome building similar features (Gemini, WebMCP)	High (strategic)	Differentiate: local memory, cross-browser, reliable hard widgets, replayable workflows; use WebMCP when present.

12. Corrections to earlier research

Some claims in earlier research documents were checked and found wrong or unsupported. Do not build on them.

Claim	Verdict
React fill snippet that calls `tracker.setValue(newValue)`	**Bug.** Makes React ignore the change. Set the value through the native setter and fire input; optionally set the tracker to the *old* value.
Antigravity uses a 'dual-engine' (BrowserMCP + Playwright CLI)	**Wrong.** That was a user's manual setup in a Google codelab. Antigravity's built-in agent uses Chrome DevTools MCP.
Per-step speed and token tables for each framework	**Unsupported.** Sources do not contain those numbers.
Anti-bot tricks (fake webdriver flag, curved mouse paths, typing jitter)	**Drop.** Not needed in the user's real browser; breaks site terms; risks store rejection.
Store run progress in `chrome.storage.session`	**Wrong for persistence.** It is wiped on restart and update. Use IndexedDB.
New Chrome Web Store rule bans CAPTCHA workarounds	**Overstated.** The August 2026 rule targets bypassing AI-platform safeguards. Avoid CAPTCHA tricks for terms-of-service reasons.
Stagehand is AGPL	**Wrong.** Stagehand is MIT; Skyvern is AGPL.
Similo failure numbers (72 vs 146)	**Refuted** in verification. Use Similo++ published accuracy (93.7% top-1) instead.
Nanobrowser = planner / navigator / validator agents	**Outdated.** Current README describes one agent on a deterministic control layer.
Desktop app can drive the user's normal Chrome via debug port	**No longer true** since Chrome 136 on the default profile.

13. Build plan

Phase	Deliverables
1. Copilot	Extension skeleton, side panel, page snapshot, summary (on-device + optional cloud), element picker, extract data from a selected region.
2. Smart fill	Encrypted vault and profile UI, AI ladder for field matching, fill engine with generic + top widget handlers, verifier, safety gates, preview/accept panel, audit log.
3. Resilient workflows	Recorder, saved steps, signatures, self-healing, IndexedDB journal and resume, stuck detection, Workday/Greenhouse/Lever adapters.

Phase	Deliverables
4. Every browser	Edge listing, Firefox desktop + Android, Safari macOS + iOS, Opera listing, bookmarklet / userscript build.
5. Expand	Pro build with real keystrokes, desktop companion, visual fallback models, WebMCP support, import/export of signed step files, enterprise install.

13.1 Measure from day one

Build a benchmark of about 50 real forms: React, Vue, Angular, plain HTML/PHP, job portals, government sites, checkouts. Track:

- Fill accuracy and **wrong-fill rate** (the most important number).
- Self-repair success rate.
- AI calls per form and cost per form.
- Median and 95th-percentile time per form.

14. Honest limits and open questions

What we cannot promise

- **Perfect fill on every site.** Widget libraries are a long tail and change. 'Ask the user' is a designed last step.
- **Full protection from prompt injection.** No one has solved it. Our design limits what an attack can reach.
- **Without the Pro build**, rare sites that only accept real keystrokes will need the user.
- **Gemini Nano** requires about 22 GB free disk and a GPU with more than 4 GB, or 16 GB RAM — many users will not have it.

14.1 Test before committing

- How well Gemini Nano maps field labels to profile fields, and its real context size.
- Whether passkey (PRF) unlock works from an extension page (Bitwarden suggests yes; keep password fallback).
- Widget handlers marked medium confidence (Chosen, react-input-mask, Inputmask, Draft.js, Quill, CKEditor, TinyMCE, react-datepicker).
- Nanobrowser internals beyond its README.
- Legal review: GDPR, site terms, Chrome Web Store Limited Use.

Appendix A. Glossary

Term	Meaning
MV3	Manifest V3 — current Chrome extension format with stricter rules.
Service worker	Extension background script; Chrome stops it when idle.
Side panel / offscreen document	Extension pages that stay alive while open; used for UI and heavy work.
Content script	Extension code injected into web pages.
Isolated / MAIN world	Content scripts normally can't see page JavaScript (isolated); MAIN world can, but the page can tamper with it.
CDP / chrome.debugger	Chrome DevTools Protocol — low-level browser control giving real keystrokes; shows a warning bar.

Term	Meaning
Accessibility (AX) tree	The browser's structured list of elements with roles and names, used by screen readers.
Shadow DOM	Encapsulated sub-trees in web components; 'closed' ones are hidden from page scripts.
WebMCP	New standard where sites describe their forms/tools for AI agents.
Gemini Nano / Prompt API	Google's small on-device model built into Chrome.
Form / field signature	Hash identifying a form or field, as Chrome autofill does.
Similo++	Published method for finding an element again by scoring many attributes.
Prompt injection	Text on a page that tries to give the AI instructions.
BYOK	Bring your own key — user supplies their own AI API key.
PRF	WebAuthn extension that lets a passkey produce an encryption key.
WXT	Build tool that produces one extension for Chrome, Edge, Firefox, Safari and Opera from one codebase.
Honeypot field	Hidden field that only bots fill in; filling it gets the form rejected.
OCR	Reading text from an image or screenshot.

Appendix B. Research files and key sources

File (in research folder)	Content
00-FINAL-client-side-blueprint.md	Decision summary.
01-research-report.md	First verified research run.
02-critic-issues-and-fixes.md	Critic review: problems and fixes.
03-cross-review-user-docs.md	Review of user-supplied research.
04-client-side-implementation-research.md	Nanobrowser, Page Agent, CDP, iframes, shadow DOM.
05-mv3-lifecycle-and-delivery.md	Chrome limits, storage, permissions, delivery options.
06-local-ai-vault-self-healing.md	On-device AI, vault, self-healing with code.
07-fill-engine-every-widget.md	Per-widget fill algorithms from library source code.
08-perception-without-aria.md	Finding controls and labels without ARIA; visual fallback.
09-edge-cases.md	~90 edge-case websites and situations with fixes.
10-lightweight-performance.md	Zero-idle-cost design and performance budget.
11-cross-browser.md	Browser support matrix, stores, mobile, fallbacks.

Key primary sources

- Chrome extension docs: developer.chrome.com/docs/extensions (service workers, storage, sidePanel, offscreen, debugger, dom, permissions, Web Store policies)
- Chrome built-in AI: developer.chrome.com/docs/ai (Prompt API, Summarizer, WebMCP)
- Nanobrowser: github.com/nanobrowser/nanobrowser · Page Agent: github.com/alibaba/page-agent · Browser Use: github.com/browser-use/browser-use
- Bitwarden: github.com/bitwarden/clients · Chromium autofill: components/autofill in chromium source
- Job fillers: github.com/berellevy/job_app_filler · github.com/andrewmillercode/Autofill-Jobs
- Self-healing: arXiv 2208.00677 (Similo), 2505.16424 (Similo++), github.com/healenium/healenium-web, Playwright selectorGenerator.ts

- Security: OWASP Password Storage Cheat Sheet; Chrome agent security guidance; Brave research on Comet prompt injection
- Local AI: github.com/mlc-ai/web-llm · huggingface.co/docs/transformers.js · docs.ollama.com